hhu

hhu
Heinrich Heine
Universität Düsseldorf

OS Development

1. Example - PC Speaker

Prof. Dr. Michael Schöttner
Dr. Fabian Ruhland

Generating Sounds

Heinrich Heine
Universität Düsseldorf

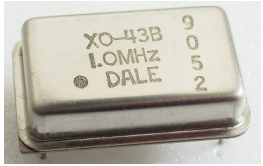
- The classic *PC Speaker* was driven by timer chip, which could generate pulses
- Timer Chip = Programmable Interval Timer (PIT)
  - Nowadays integrated inside the motherboards chipset

- On older hardware, the motherboard's firmware used the PC Speaker to generate Beep Codes in case of hardware failures
- Nowadays, many motherboards posess a hexadecimal display to show error codes →

Programmable Interval Timer

Heinrich Heine
Universität Düsseldorf

- PIT runs at a frequency of 1.19318 MHz → *Why?*
  - 1.19318 MHz = 1/4 of 4.77 MHz = Frequency of the IBM PC XT
  - Why did the XT run at exactly 4,77 MHz?
  - 4.77MHz = 1/3 of 14.31816 MHz
  - 14,31816 MHz is the base frequency needed to operate NTSC television
    → Quartz crystal for that frequency were cheap (mass production)



- Every computer uses a quartz crystal to generate its base frequency
  - This frequency is then adapted for different computer parts, using divisiors

- Nowadays, the PIT can still be used to implement a monotonic increasing system time, or to distribite CPU time in preemptive multitasking

PIT: Structure

- Three counters/channels (16 bits each)
  - Are decremented with the falling edge of the base frequency
- Usage:
  - Channel 0: Periodic interrupts → Signal OUT0 to IRQ controller
  - Channel 1: DRAM Refresh → Signal OUT1 to DMA controller (not relevant anymore)
  - Channel 2: Sound generation → OUT2 to PC Speaker
- Programming is done via *IO-ports* (8 bits each)

Port	Register	Access mode
0x40	Channel 0	read/write
0x41	Channel 1	read/write
0x42	Channel 2	read/write
0x43	Control register	only write

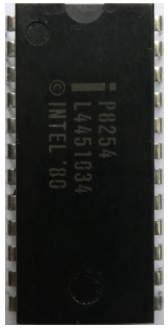
PIT: Control Register (Port 0x43)

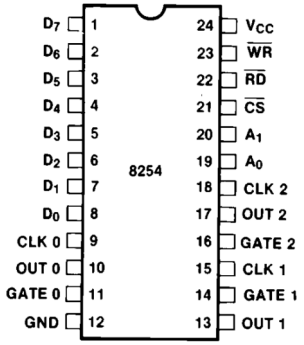
- Writing an 8-bit control word to the control register tells the PIT what to do next
- Operating mode determines how the channel operates and whether it triggers external events using its OUT signal

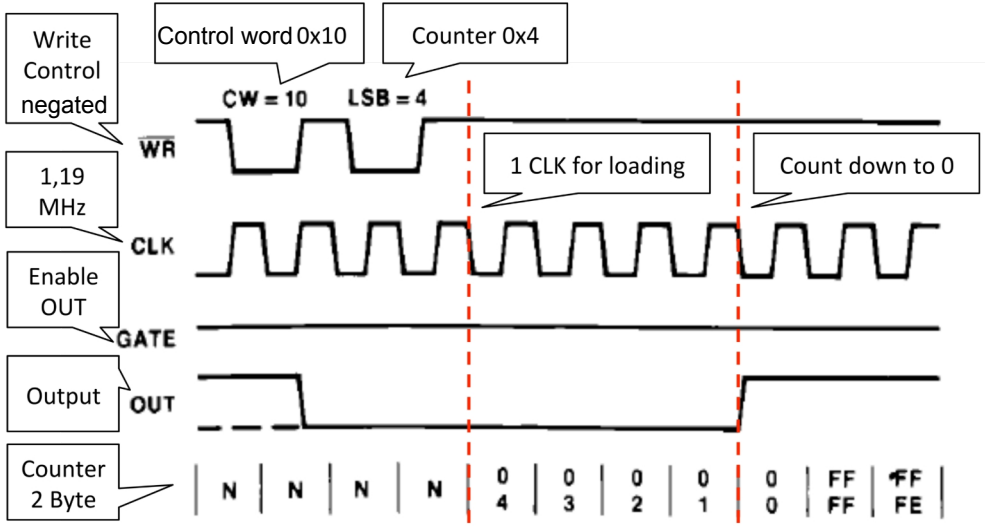
Bits	Wert	Bedeutung
6-7	**Channel Select**	
	00	Channel 0
	01	Channel 1
	10	Channel 2
	11	Invalid
4-5	**Access Mode**	
	00	Latch count value command
	01	Low counter byte
	10	High counter byte
	11	First low, then high coutner byte
1-3	**Operating Mode (0-5)**	
0	Counter format	
	0	Binary counter (16 bit)
	1	BCD counter (four digits)

PIT Mode 0: Interrupt On Terminal Count

- Counter decrements every 838ns (1 s / 1193180 Hz = 838ns)
- If the counter reaches 0, the OUTx signal is set to 1, and an interrupt is triggered
  - x = Channel number
  - The counter is not automatically reloaded in mode 0



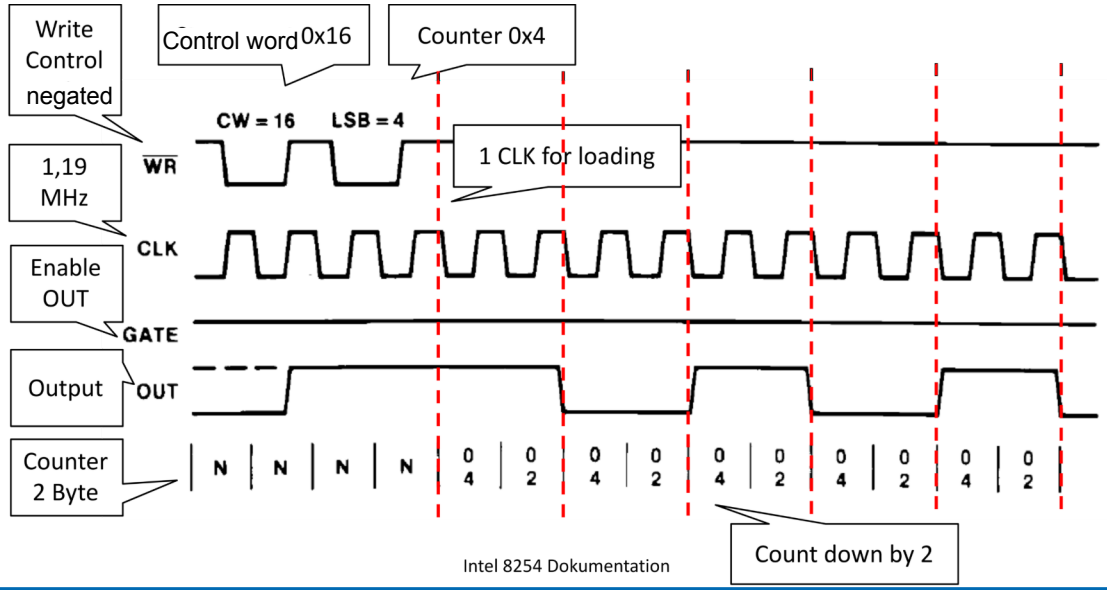




Intel 8254 Dokumentation

PIT Mode 3: Square Wave Generator

- Rectangular signal is generated using an internal Flip Flop
  - Time interval is doubled → To compensate, the counter is decremented 2x on each falling edge
- Suitable for sound generation (but also system time or preemptive multitasking)

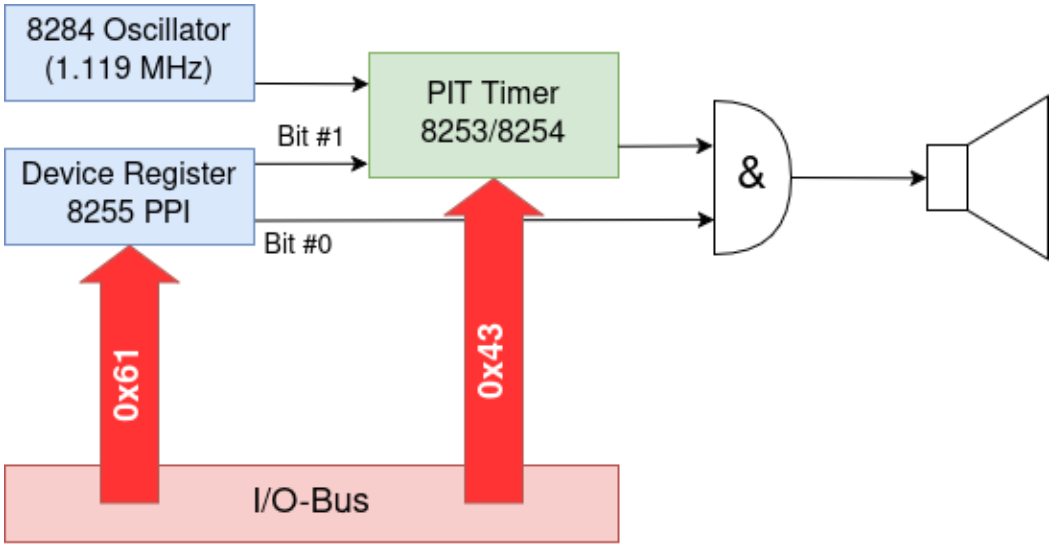


Intel 8254 Dokumentation

PC Speaker

- Programmable via IO-Ports using assembly commands: `in`, `out`

1) Channel 2 needs to be configured via the control register
   → Control word 0xB6 equals 1011 0110 in binary and means:
   - Channel 2
   - Access mode: First low counter byte, then high counter byte
   - Operating mode 3 (square wave)
   - Binary counter mode (not BCD)
2) Write start value for counter
3) Turn on the PC speaker via the Peripheral Port Interface (PPI 8255, IO-Port 0x61)



Example code in Rust

Heinrich Heine Universität Düsseldorf

```
const PORT_CTRL: u16  = 0x43;
const PORT_DATA0: u16 = 0x40;
const PORT_DATA2: u16 = 0x42;
const PORT_PPI: u16   = 0x61;

pub fn play(frequency: u32) {                          // Tone frequency in Hz
    const base_freq   = 1193180;                  // 1.19 MHz
    let counter       = base_freq / frequency;
    let status: u8;

    unsafe {
        cpu::outb(PORT_CTRL, 0xb6);               // Configure channel 2
        cpu::outb(PORT_DATA2, (counter & 0xff) as u8); // Low byte
        cpu::outb(PORT_DATA2, (counter >> 8) as u8); // High byte

        // Turn on PC speaker via PPI
        status = cpu::inb(PORT_PPI);                   // Read PPI status
        cpu::outb(PORT_PPI, status | 3); // Turn on PC speaker
    }
}
```

Why Rust?

Heinrich Heine
Universität Düsseldorf

- Type safe modern und fast programming language
- Offers compile time garbage collection
  - No null pointers
  - No „broken“ pointers (unless *unsafe* is used)
- Used by Microsoft
  - https://msrc.microsoft.com/blog/2019/07/why-rust-for-safe-systems-programming/
  - https://msrc.microsoft.com/blog/2019/07/we-need-a-safer-systems-programming-language/
- Linux allows writing kernel modules in Rust
  - https://www.heise.de/news/Rust-Code-im-Linux-Kernel-Merge-steht-laut-Linus-Torvalds-ab-Linux-5-20-bevor-7154453.html